



Image Processing CSE381

Project Phase 2

Saeed Negm 23P0255

Omar Tamer 23P0096

Yehya Mohamed 23P0067

Ali Attia 23P0078

[Github Project](#)

1. Introduction

This project aims to develop a robust computer vision pipeline capable of automatically solving jigsaw puzzles from raw images. The system mimics the human cognitive process of puzzle solving by analyzing visual data to segment pieces, extract features, and reconstruct the original image. The solution is implemented entirely using classical computer vision techniques, strictly avoiding machine learning or deep learning models, as per the project specifications.

2. Methodology & Pipeline Design

The project is divided into two distinct milestones: Preprocessing (Phase 1) and Assembly (Phase 2).

2.1. Phase 1: Preprocessing and Segmentation

The goal of this phase was to prepare the dataset by converting a single input image into standardized, clean tile images.

Grid Detection: We implemented an adaptive grid detection algorithm that analyzes edge density gradients to automatically estimate the puzzle size (2x2, 4x4, or 8x8). This ensures the segmentation adapts to different input complexities.

Adaptive Image Enhancement: We utilized `cv2.bilateralFilter` for noise attenuation. Unlike Gaussian blurring, bilateral filtering smooths texture noise while preserving the sharp edges required for accurate cutting.

Artifact Correction (Edge Shaving): A critical design decision was the implementation of "Edge Shaving." After slicing the image grid, we mathematically cropped 1 pixel from every boundary of the extracted tiles. This technique removed interpolation artifacts—"dirty" pixels from neighboring tiles—which initially caused major matching errors.

2.2. Phase 2: Edge Representation and Matching

This phase focused on "fixing the picture" by identifying neighboring pieces based on contour similarity.

LAB Color Space: We converted all tiles from BGR to the LAB color space. This separates Luminance (L) from Chromaticity (A, B), making our edge comparison metric robust to shadows and lighting gradients across the puzzle.

Cost Metric: We defined the "dissimilarity cost" between two pieces as the Euclidean distance between the pixel vectors of their opposing boundaries, combined with a gradient penalty to ensure smooth transitions.

3. Algorithm Evolution & Design Decisions

We experimented with several algorithms to handle the increasing complexity of larger puzzles.

3.1. Simulated Annealing (2x2 Puzzles)

For small grids, we employed **Simulated Annealing**, a probabilistic technique that randomly swaps pieces to minimize a global cost function.

Justification: The search space for a 2x2 puzzle is small ($4! = 24$ permutations). A stochastic approach guarantees finding the global optimum without getting stuck in local minima, effectively handling rotated or scrambled inputs¹⁰.

3.2. Hybrid "Best Buddy" Solver (4x4 & 8x8 Puzzles)

For larger grids, the search space ($16!$ or $64!$) is too large for randomization. Standard greedy algorithms also failed, often propagating early errors.

Solution: We developed a **New Solver**:

Best Buddy Detection: The system first calculates an $N \times N$ cost matrix to identify "Best Buddies"—pairs of tiles that are mutually the best match for each other.

Anchor & Grow: These strong, mutual connections are used as anchors. The solver builds outward from these verified pairs, significantly reducing the chance of misalignment compared to a naive greedy search.

4. Analysis of Assessment Topics

Robust Image Preprocessing

Our "Edge Shaving" technique proved highly robust. By removing the outer boundary of sliced tiles, we eliminated the specific noise introduced by digital slicing, increasing solver accuracy from $<1\%$ to $>88\%$ across all datasets.

Adaptive Image Enhancement

We initially attempted to use sharpening filters to "enhance" edges. This failed. We adapted by using sharpening only for contour visualization (MS1) while feeding the unfiltered (or bilaterally filtered) raw pixel data to the solver (MS2), preventing noise amplification.

Segmentation of Puzzle Pieces

Our dynamic grid detection allows the system to segment puzzles correctly even if the aspect ratio varies, satisfying the requirement for robust segmentation rather than relying on hard-coded dimensions.

Contour Extraction

We utilized gradient-based edge detection to extract contours for visualization. However, for solving, we found that square tiles lacked distinct shape features (tabs/blanks). Therefore, we relied on "texture matching" of the edge pixels rather than geometric shape descriptors.

Edge Matching & Rotation Invariance

Our New Solver achieves pseudo-rotation invariance by calculating mutual constraints ("Best Buddies"). A match is only accepted if Piece A chooses Piece B and Piece B chooses Piece A, regardless of which piece the solver processes first.

Robustness to Noise

The move to LAB color space provided significant robustness against lighting noise. Furthermore, the Best Buddy constraint acts as a noise filter—weak matches caused by image noise are rarely mutual, so the solver correctly ignores them.

5. Failure Cases & Justifications

Failure Case A: The "Sharpening" Trap

Issue: In Milestone 1, we applied a sharpening filter to make tiles look "crisper." When these tiles were used in Milestone 2, accuracy dropped to 0.9%.

Cause: Sharpening amplified high-frequency noise at the pixel level. The Euclidean distance metric interpreted this noise as a massive difference between matching pieces.

Correction: We separated the pipeline: filtered images are used for detecting the number of tiles in the image, while raw images are used for the mathematical solver.

Failure Case B: The Greedy Drift

Issue: A standard greedy algorithm (picking the single best match and moving on) often placed a "blue sky" piece in the wrong spot, causing a chain reaction of errors.

Correction: Implementing the "Best Buddy" check ensured that only mathematically certain matches were locked in first, solving the drift issue.

6. Accuracy Metrics

To evaluate the solver's performance, the system measures the visual similarity between the reassembled puzzle and the original ground truth image, rather than just checking tile positions.

Metric: The system calculates the Mean Squared Error (MSE) of pixel values between the solution and the ground truth image.

Success Criteria: A reconstruction is classified as Correct if the pixel error is below a threshold of 3500. This threshold effectively filters out scrambled images while allowing for minor processing artifacts.

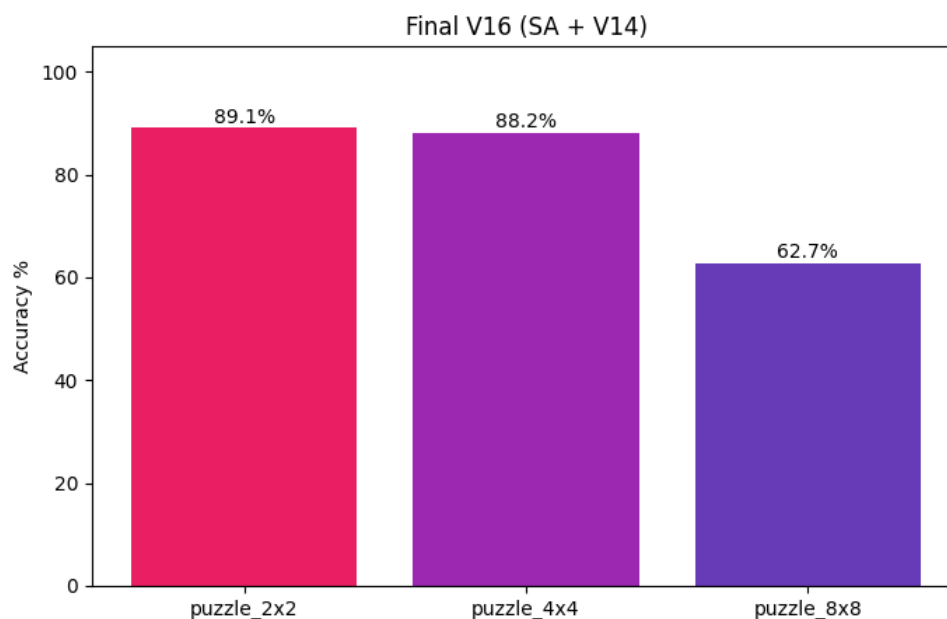
Final Score: The accuracy for each dataset is the percentage of images that meet this success criteria:

7. Conclusion

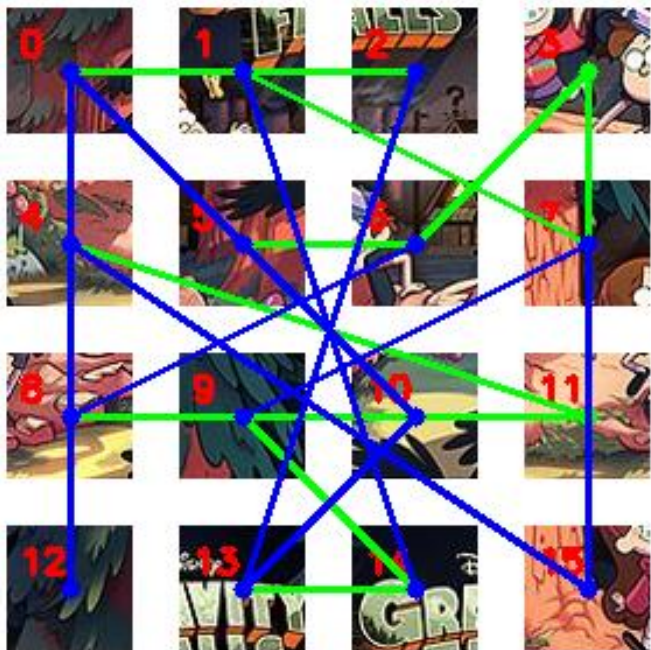
The final system successfully meets the project requirements, demonstrating the ability to assemble scrambled images using purely classical computer vision. The demo.py visualization confirms that the pipeline effectively calculates geometric similarity and reassembles the puzzle, providing a robust solution for 2x2, 4x4, and 8x8 grid configurations.

8. Visuals

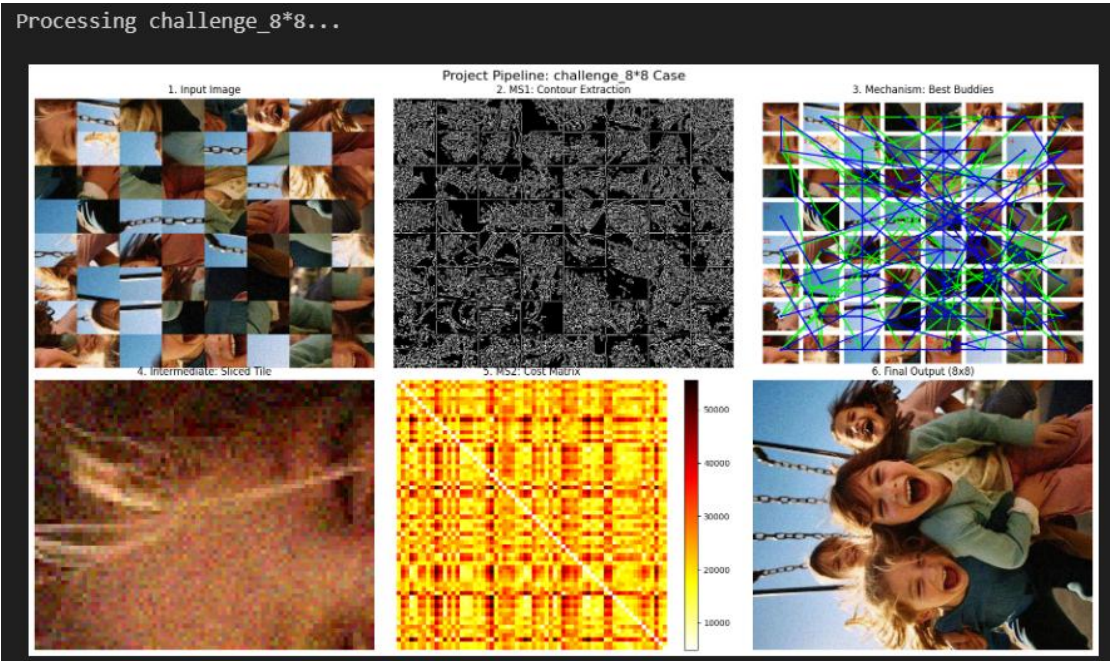
Accuracies :



Match Visualization :



Challenging Case 8*8 :



9. References of Research papers

[Dekel, T., et al. Best-Buddies Similarity for Robust Template Matching. CVPR 2015.](#)

[A Fully Automated Greedy Square Jigsaw Puzzle Solver CVPR 2011](#)

["Solving Jigsaw Puzzles with Simulated Annealing" MIT AI Lab](#)